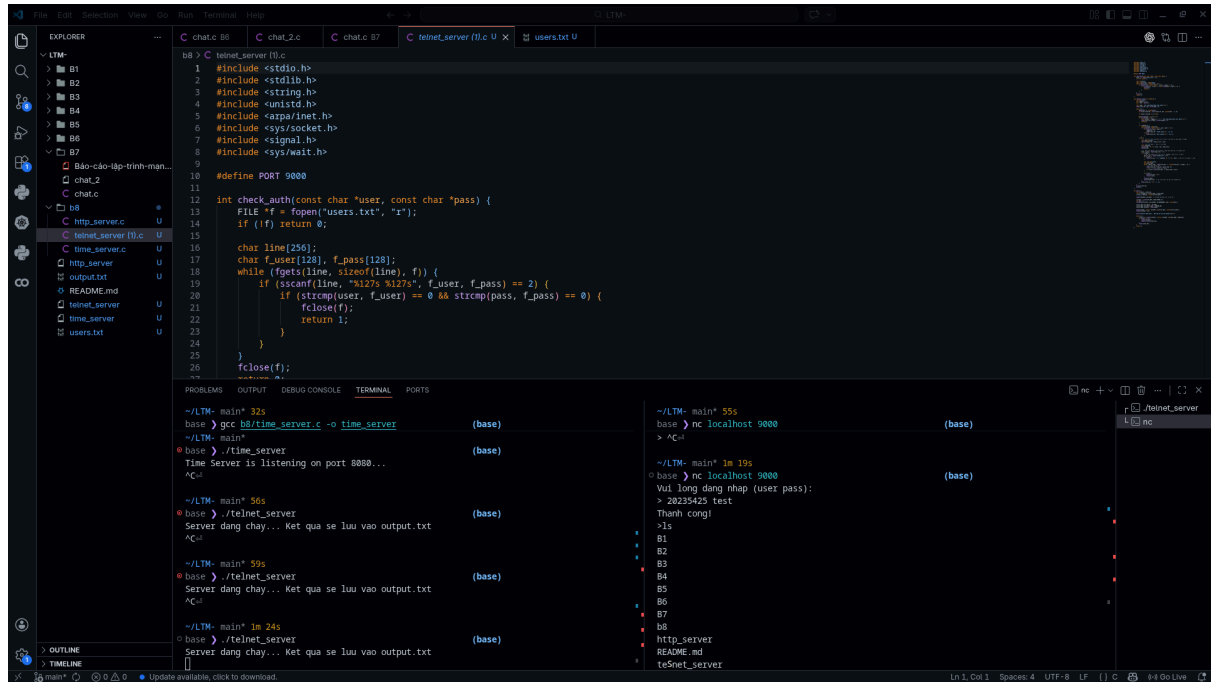


# BÁO CÁO LẬP TRÌNH MẠNG

Ngày 11 tháng 5 năm 2026

Trịnh Phú Thái 20235425

## Bài 1 : Lập trình ứng dụng telnet\_server (slide 174) bằng kỹ thuật multiprocessing



```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <string.h>
4 #include <unistd.h>
5 #include <arpa/inet.h>
6 #include <sys/socket.h>
7 #include <signal.h>
8 #include <sys/wait.h>
9
10 #define PORT 9000
11
12 int check_auth(const char *user, const char *pass) {
13     FILE *f = fopen("users.txt", "r");
14     if (!f) return 0;
15
16     char line[256];
17     char f_user[128], f_pass[128];
18     while (fgets(line, sizeof(line), f)) {
19         if (sscanf(line, "%127s %127s", f_user, f_pass) == 2) {
20             if (strcmp(user, f_user) == 0 && strcmp(pass, f_pass) == 0) {
21                 fclose(f);
22                 return 1;
23             }
24         }
25     }
26     fclose(f);
27     return 0;
28 }
29
30 int main() {
31     int sock = socket(AF_INET, SOCK_STREAM, 0);
32     if (sock < 0) {
33         perror("socket");
34         return 1;
35     }
36     struct sockaddr_in server_addr;
37     server_addr.sin_family = AF_INET;
38     inet_pton(AF_INET, "0.0.0.0", &server_addr.sin_addr);
39     server_addr.sin_port = htons(PORT);
40
41     if (bind(sock, (struct sockaddr *)&server_addr, sizeof(server_addr)) < 0) {
42         perror("bind");
43         return 1;
44     }
45
46     if (listen(sock, 5) < 0) {
47         perror("listen");
48         return 1;
49     }
50
51     while (1) {
52         struct sockaddr_in client_addr;
53         socklen_t len = sizeof(client_addr);
54         int conn = accept(sock, (struct sockaddr *)&client_addr, &len);
55         if (conn < 0) continue;
56
57         pid_t pid = fork();
58         if (pid < 0) {
59             perror("fork");
60             continue;
61         }
62         if (pid == 0) {
63             // Child process
64             close(sock);
65             if (!check_auth(user, pass)) {
66                 printf("Invalid username or password\n");
67                 exit(1);
68             }
69             printf("User %s logged in\n", user);
70             // Write to output.txt
71             FILE *out = fopen("output.txt", "a");
72             if (!out) {
73                 perror("fopen");
74                 exit(1);
75             }
76             fprintf(out, "%s\n", user);
77             fclose(out);
78             exit(0);
79         }
80         // Parent process
81         close(conn);
82     }
83     return 0;
84 }
```

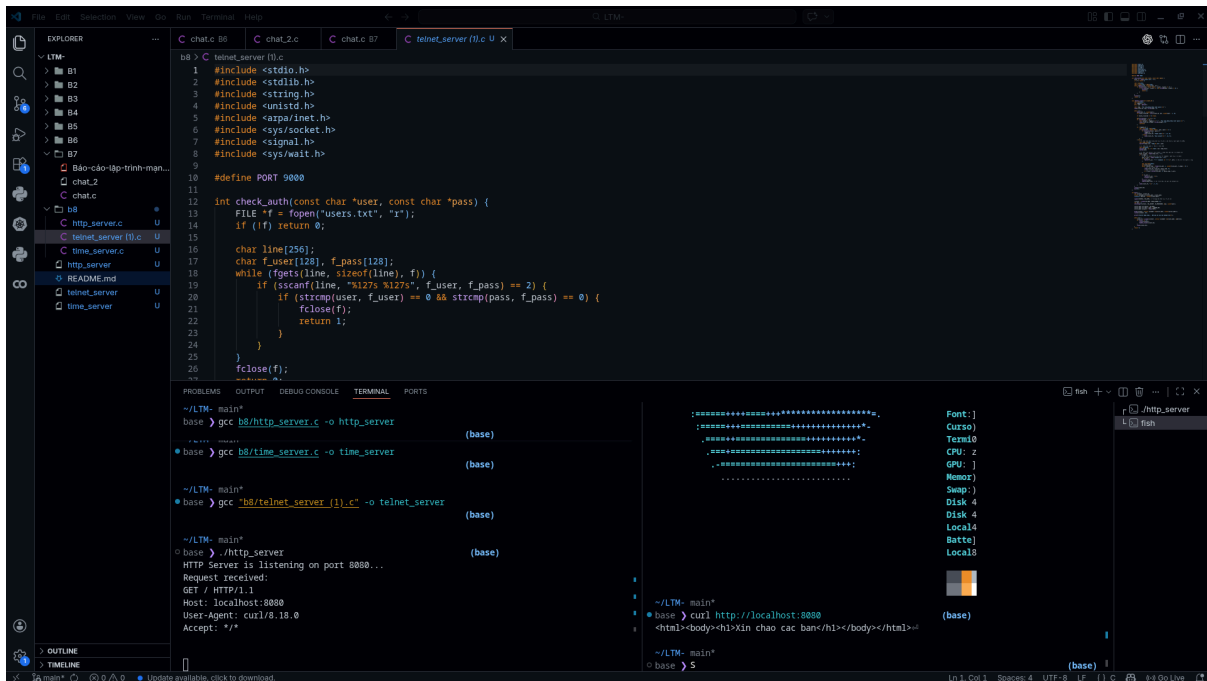
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
~/LTM-main* 32s
base > gcc b8/telnet_server.c -o telnet_server (base)
~/LTM-main*
base > ./telnet_server (base)
Time Server is listening on port 9000...
^C
~/LTM-main* 56s
base > ./telnet_server (base)
Server đang chạy... Ket qua se luu vao output.txt
^C
~/LTM-main* 59s
base > ./telnet_server (base)
Server đang chạy... Ket qua se luu vao output.txt
^C
~/LTM-main* 1m 24s
base > ./telnet_server (base)
Server đang chạy... Ket qua se luu vao output.txt
^C
```

```
~/LTM-main* 55s
base > nc localhost 9000 (base)
> ^C
~/LTM-main* 1m 19s
base > nc localhost 9000 (base)
Vui long dang nhap (user pass):
> 20235425 test
Thanh cong!
> ls
B1
B2
B3
B4
B5
B6
B7
B8
http_server
README.md
telnet_server
teNet_server
```

Người dùng nhập đúng username password ( được cấp ở users.txt) và dùng đúng lệnh thì sẽ hiển thị kết quả tương ứng và lưu vào output.txt

## Bài 2 : Lập trình ứng dụng http\_server (slide 154 ) sử dụng kỹ thuật preforking



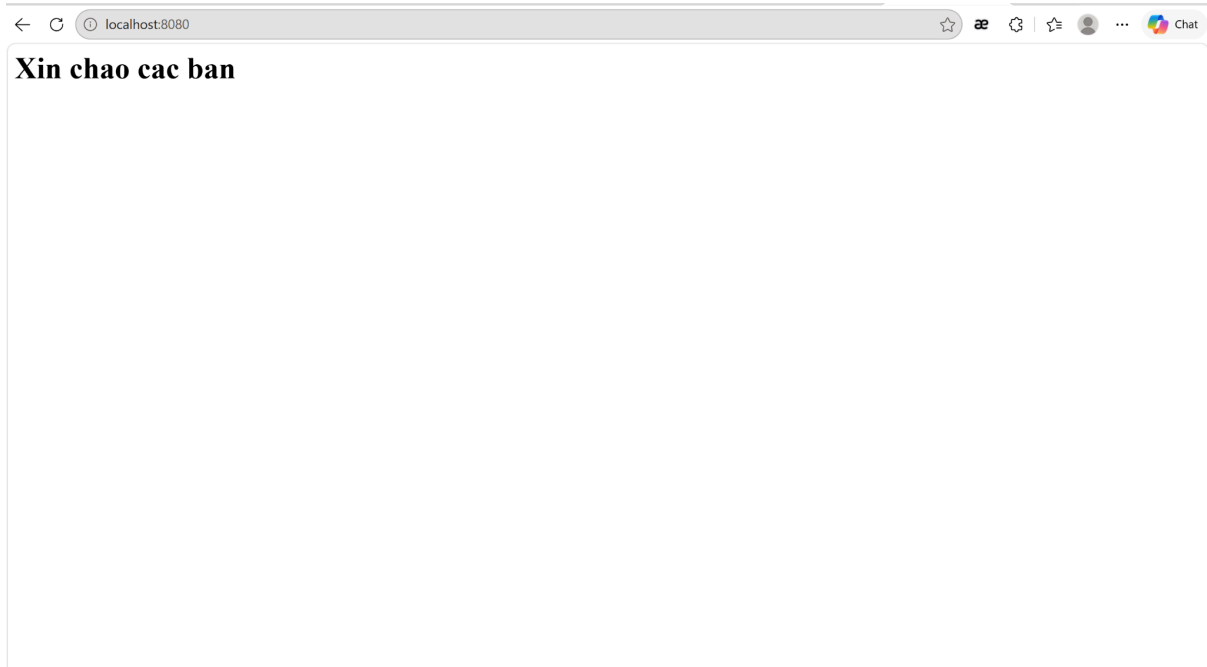
```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <string.h>
4 #include <unistd.h>
5 #include <arpa/inet.h>
6 #include <sys/socket.h>
7 #include <signal.h>
8 #include <sys/wait.h>
9
10 #define PORT 8080
11
12 int check_auth(const char *user, const char *pass) {
13     FILE *f = fopen("users.txt", "r");
14     if (!f) return 0;
15
16     char line[256];
17     char f_user[128], f_pass[128];
18     while (fgets(line, sizeof(line), f)) {
19         if (sscanf(line, "%127s %127s", f_user, f_pass) == 2) {
20             if (strcmp(user, f_user) == 0 && strcmp(pass, f_pass) == 0) {
21                 fclose(f);
22                 return 1;
23             }
24         }
25     }
26     fclose(f);
27     return 0;
28 }
```

```
~/LTM- main*
base > gcc b8/http_server.c -o http_server
(base)
base > gcc b8/time_server.c -o time_server
(base)
~/LTM- main*
base > gcc "b8/telnet_server (1).c" -o telnet_server
(base)
~/LTM- main*
base > ./http_server
HTTP Server is listening on port 8080...
Request received:
GET / HTTP/1.1
Host: localhost:8080
User-Agent: curl/8.18.0
Accept: */*

~/LTM- main*
base > curl http://localhost:8080
<html><body><h1>Xin chao cac ban</h1></body></html>

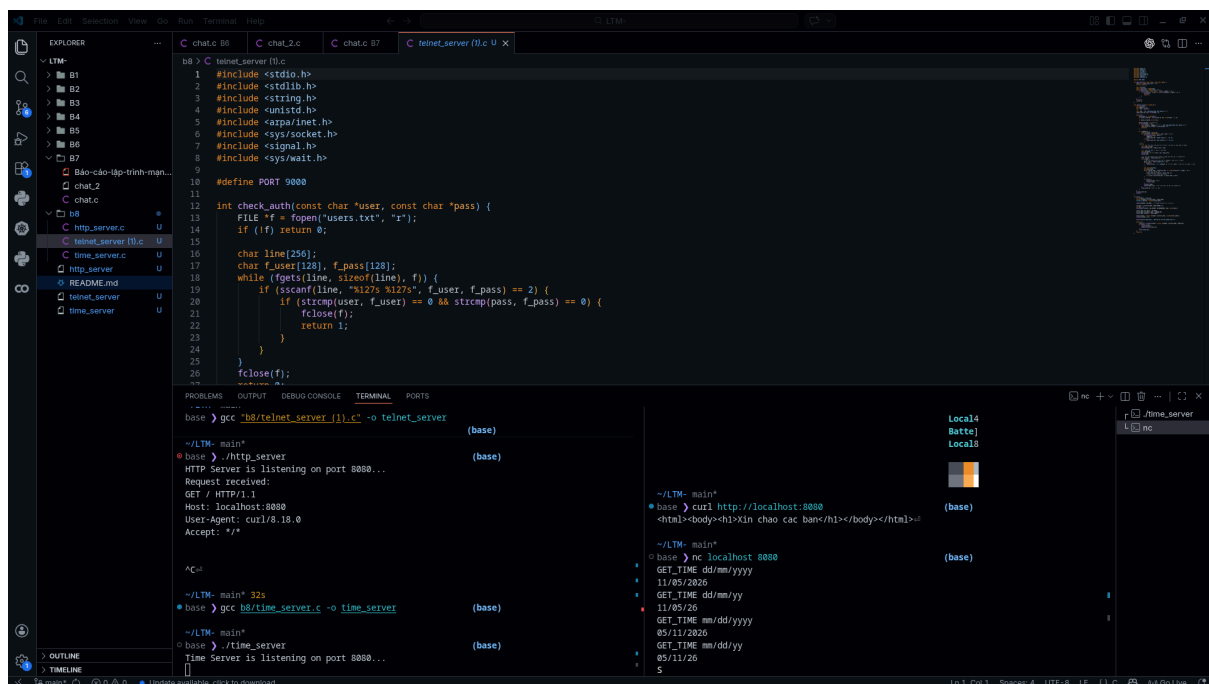
~/LTM- main*
base > S
(base)
```

biên dịch và chạy server, khi người dùng chạy [localhost:8080](http://localhost:8080) thì sẽ hiện



### Bài 3

- Lập trình ứng dụng **time\_server** thực hiện chức năng sau:
  - Chấp nhận nhiều kết nối từ các client sử dụng kỹ thuật multiprocessing.
  - Client gửi lệnh GET\_TIME [format] để nhận thời gian từ server.
  - **format** là định dạng thời gian server cần trả về client. Các format cần hỗ trợ gồm:
    - dd/mm/yyyy – vd: 30/01/2023
    - dd/mm/yy – vd: 30/01/23
    - mm/dd/yyyy – vd: 01/30/2023
    - mm/dd/yy – vd: 01/30/23
  - Cần kiểm tra tính đúng đắn của lệnh client gửi lên server.



```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <string.h>
4 #include <unistd.h>
5 #include <sys/inet.h>
6 #include <sys/socket.h>
7 #include <signal.h>
8 #include <sys/wait.h>
9
10 #define PORT 9000
11
12 int check_auth(const char *user, const char *pass) {
13     FILE *f = fopen("users.txt", "r");
14     if (!f) return 0;
15
16     char line[256];
17     char f_user[128], f_pass[128];
18     while (fgets(line, sizeof(line), f)) {
19         if (sscanf(line, "%127s %127s", f_user, f_pass) == 2) {
20             if (strcmp(user, f_user) == 0 && strcmp(pass, f_pass) == 0) {
21                 return 1;
22             }
23         }
24     }
25     fclose(f);
26     return 0;
27 }
```

```
base > gcc "b8/telnet_server (1).c" -o telnet_server
~/LTM- main*
base > ./http_server
HTTP Server is listening on port 8080...
Request received:
GET / HTTP/1.1
Host: localhost:8080
User-Agent: curl/8.18.0
Accept: */*
^C
~/LTM- main* 32s
base > gcc b8/time_server.c -o time_server
~/LTM- main*
base > ./time_server
Time Server is listening on port 8080...
~/LTM- main*
base > curl http://localhost:8080
<html><body><h1>xin chào các bạn</h1></body></html>
~/LTM- main*
base > nc localhost 8080
GET_TIME dd/mm/yyyy
11/05/2026
GET_TIME dd/mm/yy
11/05/26
GET_TIME mm/dd/yyyy
05/11/2026
GET_TIME mm/dd/yy
05/11/26
5
```